

DC Lab — Experiment 8 Walkthrough (General Purpose)

Redis Caching for Fault Tolerance (Cache-Aside Pattern)

Goal: Add a caching layer in front of an existing backend service so that (a) repeated reads are served fast without hitting the backend every time, and (b) the system stays available even when either the cache or the backend fails — it should degrade gracefully instead of returning an error.

How this connects to earlier experiments: This experiment doesn't stand alone — it sits directly on top of the gRPC service and Lamport clock work from Experiment 2/3. The “database” here is literally the same backend server (`server.py`) built in Exp 2, reused as-is with **zero code changes**. Exp 8 adds a new layer *in front of* it, the same way a real system evolves: you don't rebuild the database, you add caching around it. If your earlier experiments used a different service name/RPC, substitute those in below — the caching logic itself is completely generic.

Prerequisite: A working gRPC backend service from an earlier experiment (anything with a simple “fetch by ID” RPC works) and an activated `venv` .

Step 1 — Install and start Redis

```
brew install redis
brew services start redis
```

Verify it's running:

```
redis-cli ping
```

Should reply `PONG` .

Step 2 — Install the Python Redis client

```
source venv/bin/activate
pip install redis
```

```
echo "redis==5.0.1" >> requirements.txt
```

Verify Python can reach Redis:

```
python -c "import redis; r = redis.Redis(); print(r.ping())"
```

Should print `True`.

Step 3 — Understand the design before writing code

Cache-aside (a.k.a. lazy loading) is the core pattern:

1. On a read, check the cache first
2. **Cache hit** → return the cached value immediately, never touch the backend
3. **Cache miss** → fetch from the backend, store the result in the cache (with a short expiry, or "TTL"), then return it

Two separate fault-tolerance paths matter here, and they're not the same thing:

- **Cache (Redis) is down:** the gateway should catch that failure and just go straight to the backend, as if there were no cache at all. The client never sees an error — they just don't get the speed benefit this one time.
- **Backend (the real service/database) is down:** this is the more interesting case. If we have *any* previously-cached value for that key — even an expired one — serving that **stale** data is usually better than returning an error. This needs a **second, separate cache key that never expires**, kept purely as a fallback-of-last-resort, distinct from the normal short-TTL cache entry.

Step 4 — Write `cache_gateway.py`

```
import json
import grpc
import redis

import your_pb2          # replace with your actual generated pb2 module
import your_pb2_grpc     # replace with your actual generated pb2_grpc module

REDIS_HOST = "localhost"
REDIS_PORT = 6379
CACHE_TTL_SECONDS = 10          # short TTL so hits/misses are easy to obse
```

```

BACKEND_SERVICE_ADDR = "localhost:50051"

STALE_KEY_PREFIX = "stale:item:"      # kept indefinitely as a fallback, never ex
FRESH_KEY_PREFIX = "fresh:item:"      # normal cache entry, expires after CACHE_T

def get_redis():
    """Fresh connection per call, with a short timeout, so we can
    detect Redis being down quickly instead of hanging."""
    return redis.Redis(host=REDIS_HOST, port=REDIS_PORT,
                       socket_connect_timeout=1, socket_timeout=1)

def fetch_from_backend(item_id):
    """Direct call to the real backend service (your earlier experiment's server)
    with grpc.insecure_channel(BACKEND_SERVICE_ADDR) as channel:
        stub = your_pb2_grpc.YourServiceStub(channel)
        response = stub.GetItem(your_pb2.ItemRequest(item_id=item_id), timeout=2)
        return {"item_id": response.item_id, "data": response.data}

def get_item(item_id):
    fresh_key = f"{FRESH_KEY_PREFIX}{item_id}"
    stale_key = f"{STALE_KEY_PREFIX}{item_id}"

    # --- Try the cache first ---
    try:
        r = get_redis()
        cached = r.get(fresh_key)
        if cached:
            print(f"[Gateway] CACHE HIT for item {item_id}")
            return json.loads(cached), "cache-hit"
        print(f"[Gateway] CACHE MISS for item {item_id} -- querying backend")
    except redis.exceptions.RedisError as e:
        print(f"[Gateway] REDIS UNAVAILABLE ({e}) -- falling back to backend dire
        r = None

    # --- Cache miss, or Redis itself is down: go to the real backend ---
    try:
        data = fetch_from_backend(item_id)
        print(f"[Gateway] Fetched item {item_id} from BACKEND")
        if r is not None:
            try:
                r.set(fresh_key, json.dumps(data), ex=CACHE_TTL_SECONDS)
                r.set(stale_key, json.dumps(data))    # no expiry - fault-toleranc
            except redis.exceptions.RedisError:
                print("[Gateway] Redis write failed, continuing without caching t

```

```

        return data, "backend"

except grpc.RpcError as e:
    # --- Backend unreachable: fall back to a stale cached copy if we have one
    print(f"[Gateway] BACKEND UNAVAILABLE ({e.code()}) -- checking for stale")
    if r is not None:
        try:
            stale = r.get(stale_key)
            if stale:
                print(f"[Gateway] Serving STALE cached copy for item {item_id}")
                return json.loads(stale), "stale-fallback"
        except redis.exceptions.RedisError:
            pass
    raise RuntimeError(f"Item {item_id} unavailable: both backend and cache f")

if __name__ == "__main__":
    import sys
    item_id = int(sys.argv[1]) if len(sys.argv) > 1 else 1
    data, source = get_item(item_id)
    print(f"\nResult (source={source}): {data}")

```

The whole strategy lives in `get_item()` — everything else is just wiring. Read it top to bottom: try cache → on miss/Redis-down go to backend → on backend failure, try the never-expiring stale key → only if *both* have failed, actually raise an error.

Step 5 — Run the demo (needs 2 terminals + Redis already running)

Terminal 1 — start your existing backend service (whatever you built in an earlier gRPC experiment):

```

source venv/bin/activate
python server.py

```

Terminal 2 — exercise the gateway

First call — should be a cache miss:

```
python cache_gateway.py 1
```

Second call, right away — should be a cache hit (and notice Terminal 1 does **not** log a

second request — proof the backend was never touched):

```
python cache_gateway.py 1
```

Step 6 — Test fault-tolerance path 1: Redis down → falls back to backend

In a third terminal:

```
brew services stop redis
```

Back in Terminal 2:

```
python cache_gateway.py 1
```

Expect `REDIS UNAVAILABLE` in the log, followed by a normal backend fetch — result is still correct, just uncached this time.

Step 7 — Test fault-tolerance path 2: backend down → falls back to stale cache

Restart Redis first:

```
brew services start redis  
redis-cli ping
```

Run the gateway once more (with both Redis and the backend up) to make sure a fresh stale-backup entry exists:

```
python cache_gateway.py 1
```

Now kill the backend (`Ctrl+C` in Terminal 1), and run the gateway again:

```
python cache_gateway.py 1
```

Expect `BACKEND UNAVAILABLE` , followed by `Serving STALE cached copy` — the client still gets a valid (if slightly old) result instead of an error.

What to check across the whole run

You should be able to point to all three outcomes in your logs:

1. `cache-hit` — fast path, backend never touched
 2. `backend` — normal cache-aside miss, or the Redis-down fallback (both land here, since either way the backend ends up serving the request)
 3. `stale-fallback` — backend-down fallback, served from the never-expiring backup key
-

Common mistakes

Mistake	Symptom	Fix
Only using one cache key per item	No way to fall back to a stale value once the fresh key expires or backend goes down	Keep two keys: a short-TTL “fresh” key and a never-expiring “stale” backup key, written together on every successful backend fetch
Not catching <code>redis.exceptions.RedisError</code> separately from backend errors	A Redis outage crashes the whole gateway instead of gracefully falling back	Wrap Redis calls in their own try/except, distinct from the gRPC call’s try/except
Setting a long TTL during testing	Hard to actually observe cache misses happening	Use a short TTL (5–10s) while developing/demoing, so you can watch entries expire in real time
	A dead	

Reusing one Redis connection across the whole gateway's lifetime	connection can hang instead of failing fast when Redis actually goes down	Create a fresh Redis client per call with a short <code>socket_connect_timeout / socket_timeout</code>
Forgetting to restart the backend before testing again	Confusing "still down" errors that aren't about caching at all	Double check Terminal 1's actual state before attributing an error to the cache layer